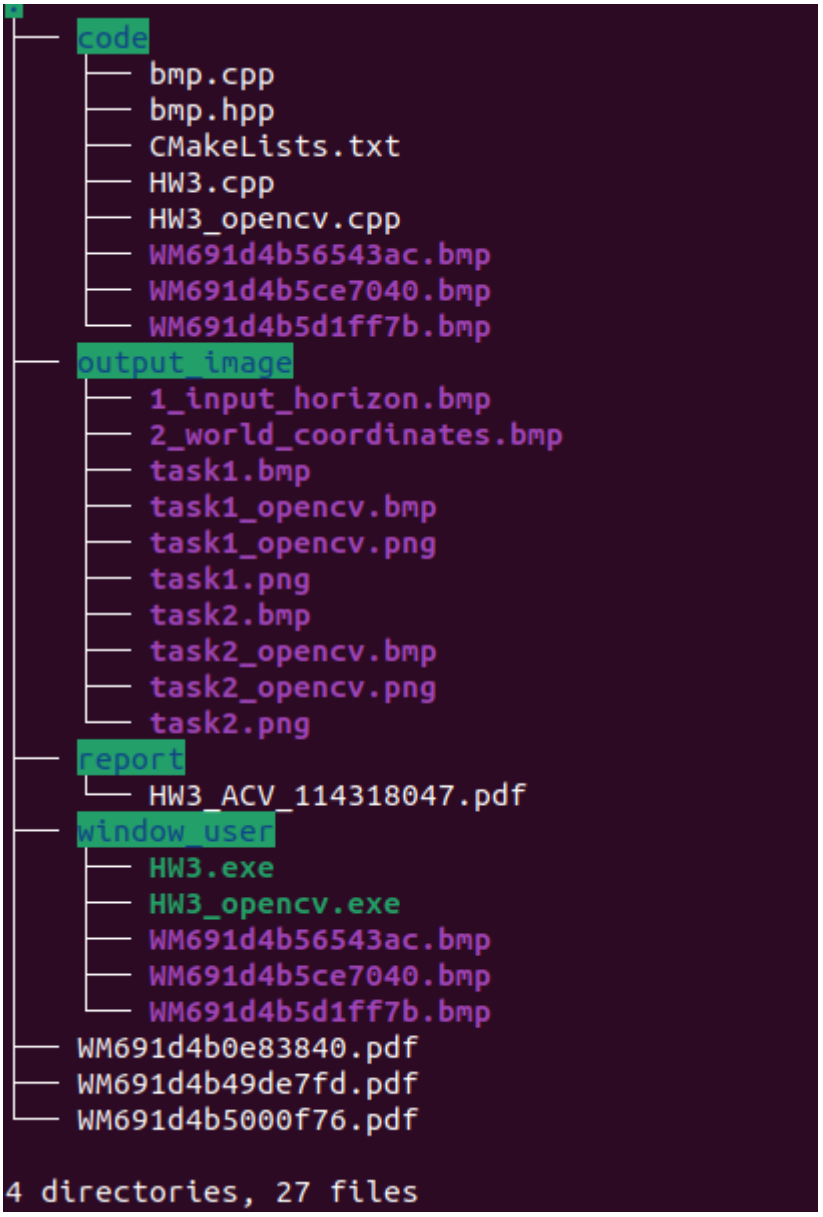
高等電腦視覺

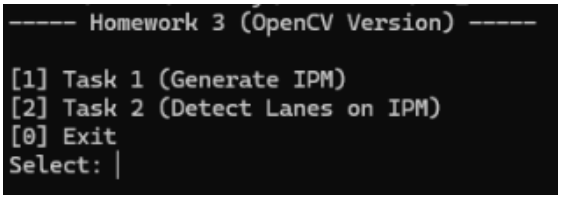
作業#(3)

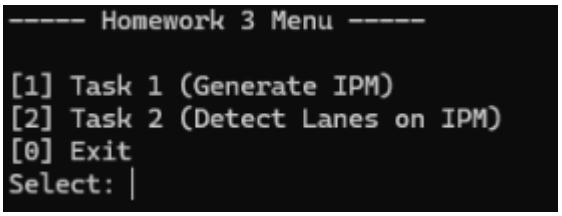
姓名:闕楷宸
學號:114318047
指導老師:黃正民

作業說明:

作業架構:


.
code
bmp.cpp
bmp.hpp
CMakeLists.txt
HW3.cpp
HW3_opencv.cpp
WM691d4b56543ac.bmp
WM691d4b5ce7040.bmp
WM691d4b5d1ff7b.bmp
output_image
1_input_horizon.bmp
2_world_coordinates.bmp
task1.bmp
task1_opencv.bmp
task1_opencv.png
task1.png
task2.bmp
task2_opencv.bmp
task2_opencv.png
task2.png
report
HW3_ACV_114318047.pdf
window_user
HW3.exe
HW3_opencv.exe
WM691d4b56543ac.bmp
WM691d4b5ce7040.bmp
WM691d4b5d1ff7b.bmp
WM691d4b0e83840.pdf
WM691d4b49de7fd.pdf
WM691d4b5000f76.pdf
4 directories, 27 files





主程式HW3.cpp HW3_opencv.cpp

輸入0~2即可輸出結果

建置執行(window,linux)

Linux:

1.使用opencv 4.5.4(sudo apt install libopencv-dev)

2.建置與執行:

cd HW#6_114318047/code/
cmake -S . -B build
cmake --build build -j
./build/HW3
./build/HW3_opencv

Window:

cd window_user
./HW3.exe
./HW3_opencv.exe

Window 建置:

1:Cmake:

winget install Kitware.CMake

2:OpenCV for window:

C:\opencv\build\x64\vc16\bin(make sure DLLs are in this dir)

3:Confirm CMake config file exists:

C:\opencv\build\x64\vc16\lib\OpenCVConfig.cmake

4:Build and run

cd HW3_114318047\code

5:Configure:

cmake -S . -B build -G "Visual Studio 17 2022" -A x64 -DOpenCV_DIR="C:\opencv\build\x64\vc16\lib"

6:Produce exe:

cmake --build .\build --config Release

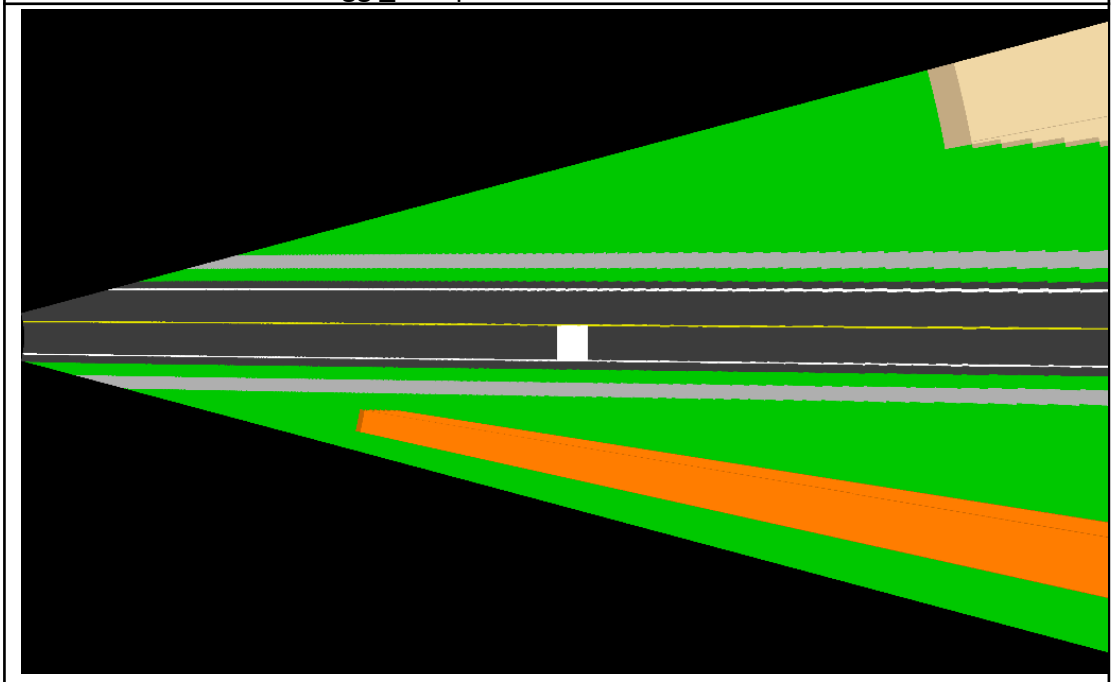
7:Run exe

.\build\Release\HW3.exe

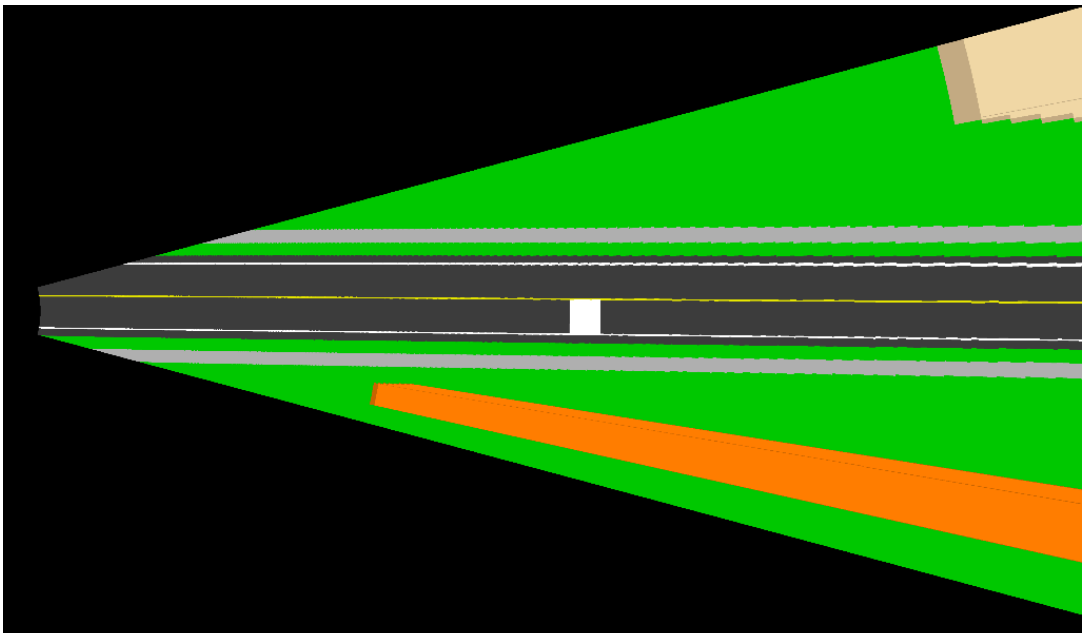
.\build\Release\HW3_opencv.exe

1. Transfer the road image to bird's eye view by inverse perspective mapping

Refer to "BertozziAndBroggi_IPM.pdf"



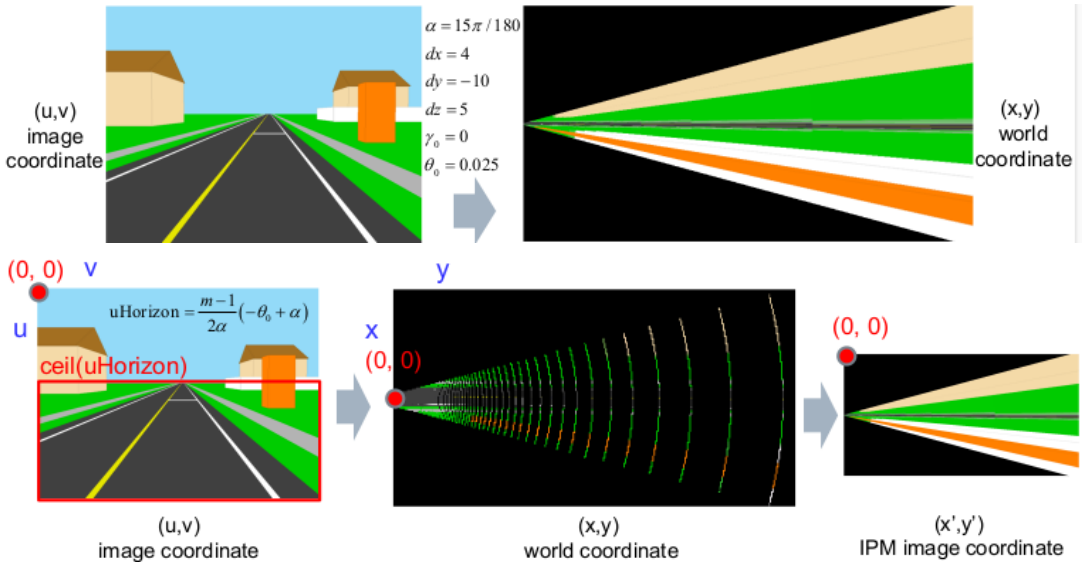
task1.bmp(task1.png)



task1_opencv.bmp(task1_opencv.png)

Discussion(1)

為了將image coordinate(u,v)映射至world coordinate(x,y)(Forward warping)，使用以下方法：




```
const float alpha  = 15.0f * M_PI / 180.0f;
const float dx     = 4.0f;
const float dy     = -10.0f;
const float dz     = 5.0f;
const float gamma0 = 0.0f;
const float theta0 = 0.025f;

const float m1 = float(H - 1);
const float n1 = float(W - 1);

// Follow the steps in the slide(transfer the pixels (u,v) in [uHorizon:Height-
// by foward warping(Image to World) to get the range [xmin:xmax,ymin:ymax] of
float uHorizon = m1 * (-theta0 + alpha) / (2.0f * alpha);
int   uStart   = int(std::ceil(uHorizon)); // round up to next integer
```

以上為參數設定。

以下為映射方法：

$$X(u,v) := d_z \cdot \cot\left[\theta_0 - \alpha + u \cdot \left(\frac{2\cdot\alpha}{m-1}\right)\right] \cdot \sin\left[\gamma_0 - \alpha + v \cdot \left(\frac{2\cdot\alpha}{n-1}\right)\right] + d_x$$

(Image to World)

$$Y(u,v) := d_z \cdot \cot\left[\theta_0 - \alpha + u \cdot \left(\frac{2\cdot\alpha}{m-1}\right)\right] \cdot \cos\left[\gamma_0 - \alpha + v \cdot \left(\frac{2\cdot\alpha}{n-1}\right)\right] + d_y$$

以上公式出自講義"Test of Bertozzi and Broggi's Inverse Persepective Mapping Functinos"，以下進行簡化，方便程式撰寫：

$$\phi = \theta_0 - \alpha + 2\alpha \frac{u}{m-1}$$

$$\psi = \gamma_0 - \alpha + 2\alpha \frac{v}{n-1}$$

$$X = \frac{dz}{\tan(\phi)} \sin(\psi) + dx$$

$$Y = \frac{dz}{\tan(\phi)} \cos(\psi) + dy$$

```
for (int u = uStart; u < H; ++u)
{
    float u_norm = float(u) / float(H - 1);
    float phi    = theta0 - alpha + 2.0f * alpha * u_norm;
    float tanphi = std::tan(phi);
    if (std::fabs(tanphi) < 1e-6f) continue;

    float common = dz / tanphi;

    for (int v = 0; v < W; ++v)
    {
        float v_norm = float(v) / n1;
        float psi    = gamma0 - alpha + 2.0f * alpha * v_norm;

        float X = common * std::sin(psi) + dx;
        float Y = common * std::cos(psi) + dy;
```

以上為將(u,v)映射到(x,y) equation

$$X(u,v) := d_z \cdot \cot\left[\theta_0 - \alpha + u \cdot \left(\frac{2 \cdot \alpha}{m-1}\right)\right] \cdot \sin\left[\gamma_0 - \alpha + v \cdot \left(\frac{2 \cdot \alpha}{n-1}\right)\right] + d_x$$

$$Y(u,v) := d_z \cdot \cot\left[\theta_0 - \alpha + u \cdot \left(\frac{2 \cdot \alpha}{m-1}\right)\right] \cdot \cos\left[\gamma_0 - \alpha + v \cdot \left(\frac{2 \cdot \alpha}{n-1}\right)\right] + d_y$$

(Image to World)

```cpp
static void world_to_image(float X, float Y, float &u, fl
{
    float x0 = X - dx;
    float y0 = Y - dy;

    // R is the distance from camera center to point's pr
    float R  = std::sqrt(x0 * x0 + y0 * y0);
    if (R < 1e-6f) { u = v = -1.0f; return; }

    // Calculate angles
    float phi = std::atan(dz / R);
    float psi = std::atan2(x0, y0);

    u = (m1 / (2.0f * alpha)) * (phi - theta0 + alpha);
    v = (n1 / (2.0f * alpha)) * (psi - gamma0 + alpha);
}
```

$\Phi = tan^{-1}(\frac{dz}{R})$

Where $R = \sqrt{(X - d_x)^2 + (Y - d_y)^2}$

$\psi = tan^{-1}((X - d_x), Y - d_y))$

$$u = \frac{m-1}{2\alpha}(\phi - \theta_0 + \alpha)$$

$$v = \frac{n-1}{2\alpha}(\psi - \gamma_0 + \alpha)$$

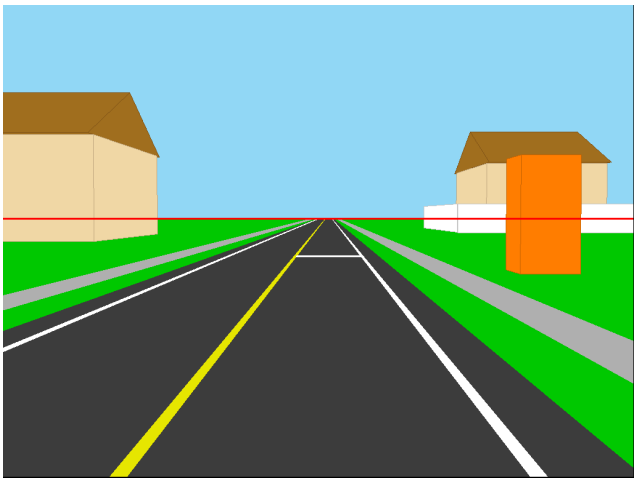
以上式子是用來簡化以下world-to-image式子

$$U(x,y) := \left(\frac{m-1}{2\cdot\alpha}\right)\cdot\left(\operatorname{atan}\left(\frac{d_z\cdot\sin\left(\operatorname{atan}\left(\frac{x-d_x}{y-d_y}\right)\right)}{x-d_x}\right) - \theta_0 + \alpha\right)$$

(World to Image)

$$V(x,y) := \left(\frac{n-1}{2\cdot\alpha}\right)\cdot\left(\operatorname{atan}\left(\frac{x-d_x}{y-d_y}\right) - \gamma_0 + \alpha\right)$$

以下為仿照講義中的步驟：



1_input_horizon.bmp

紅線以下為uHorizon

下圖為world coordinate圖像：

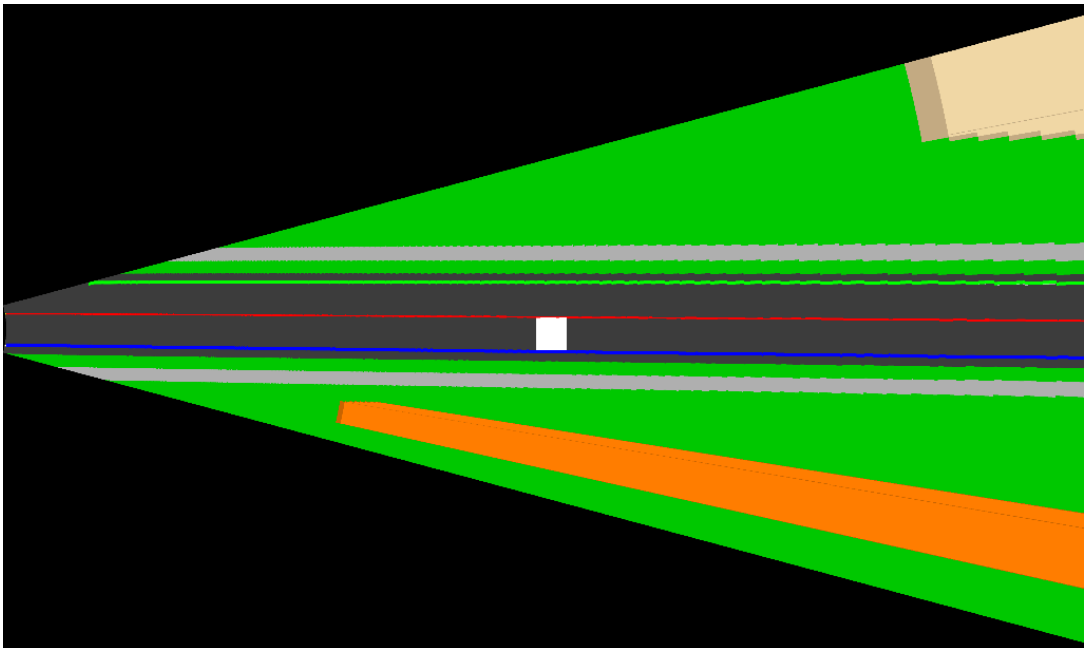
2_world_coordinates.bmp

上圖為word coordinate的圖像，之所以每一個扇形的間距會這麼大是因為forward warping的問題是原圖的pixel有限，因此對造成映射後會產生黑色空隙，在附件pdf中也有提到此問題

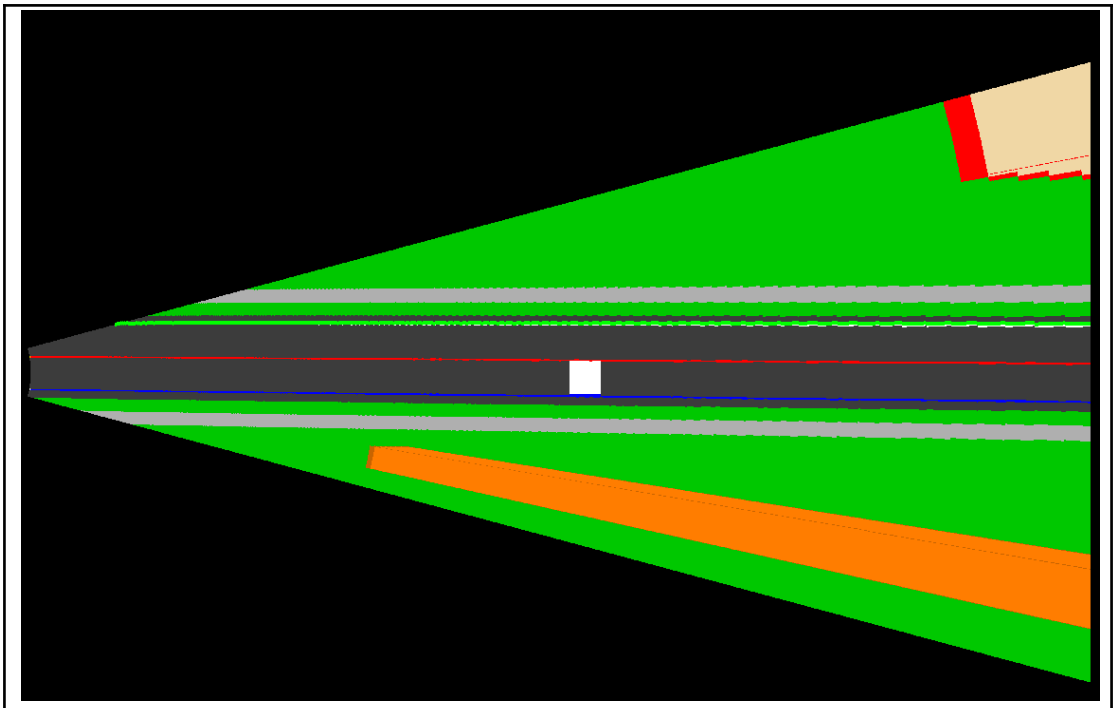
The transformations do get the general perspective effects correct. Image points map to world points which fan out in the x direction and spread out in the y direction, as expected. World points map to im

2.Detect the edge points of the lanes in the road image

Label three lanes as blue,red and green respectively



task2.bmp(task2.bmp)



task2_opencv.bmp(task2_opencv.png)

Discussion (2)

此題需要在IPM影像中將路上的路標線進行標注，有兩條白色的標線以及中間一條黃色得標現所組成沒有使用常見的邊緣檢測方法進行處理，而是單靠顏色進行標注及塗色，因此當路標線不夠清楚時會有失敗的可能，而實際標注的方法如下：

1. 在IPM圖像中逐行(x)檢查

```
for (int x = 0; x < W; ++x)
```

2. 在該行(x)中檢查是否為白，如果是，將pixel設為綠色或藍色

```
bool isWhite = (r > 200 && g > 200 && b > 200);
```

3. 在該行(x)中檢查是否為黃，如果是將該pixel設為紅色

```
if (r > 180 && g > 180 && b < 150)
```

此方法操作簡單，速度快，不須計算梯度、邊緣等計算，但是在模糊的圖像中效果受限，如果使用hough transform可以增強檢查線段的能力。